Source Code and Output

Code:

```
#include <iostream>
#include <cstdint>

using namespace std;

class Node{
    public:
    int data;
    int next;
    Node (): next(-1){}
    Node (int value): data(value), next(-1){}
    void display(){
        cout << "[" << data << "|" << next << "]";
    }
};


class StaticList{
    public:
    int list;
    int avail;
    Node* array;
    StaticList(int MAX_SIZE = 10){
        array = (new Node[10]);
        list = -1;
        avail = 0;
        for (int i=0; i<MAX_SIZE-1; i++){
            array[i].next = i+1;
        }
        array[MAX_SIZE-1].next = -1;
    }
    ~StaticList(){delete array;}
    int get_node(int _data){
        if (avail == -1){
            cout << "List Overflow!!" << endl;
            return -1;
        }
        int index = avail;
        array[index].data = _data;
        // cout << "Avail=" << avail << endl;
        avail = array[avail].next;
        // cout << "Avail=" << avail << endl;
        return index;
    }
    void free_node(int index){
        array[index].next = avail;
        avail = index;
```

```
    }
    void traverse_display_active_list(){
        cout << "Active List: " << endl;
        if (list == -1){
            cout << "[-1|-1]" << endl;
        }
        else{
            int idx = list;
            while (array[idx].next != -1){
                array[idx].display();
                idx = array[idx].next;
                cout << "--->";
            }
            array[idx].display();
            cout << endl;
        }
        cout << "----------------------------" << endl;
    }
    void traverse_display_free_list(){
        cout << "Free List: " << endl;
        if (avail == -1){
            cout << "[-1|-1]" << endl;
        }
        else{
            int idx = avail;
            while (array[idx].next != -1){
                array[idx].display();
                idx = array[idx].next;
                cout << "--->";
            }
            array[idx].display();
            cout << endl;
        }
        cout << "................................." << endl;
    }

    int ins_beg(int value){
        int index = get_node(value);
        if (index == -1){
            exit(1);
        }
        // cout << "index=" << index << endl;
        array[index].next = list;
        // cout << "array[index].next=" << array[index].next << endl;
        list = index;
        cout << "Just inserted at beginning: " << endl;
        array[index].display();
        cout << endl;
        return 0;
    }
    int ins_end(int value){
        int index = get_node(value);
        if (index == -1){
            return -1;
```

```
        }
        int idx = list;
        if (idx == -1){
            list = index;
            array[index].next = -1;
            return 0;
        }
        while (array[idx].next != -1){
            idx = array[idx].next;
        }
        array[idx].next = index;
        cout << "Just inserted at end: " << endl;
        array[index].display();
        cout << endl;
        array[index].next = -1;
        return 0;
    }

    int ins_specific(int value, int target){
        int index = get_node(value);
        if (index == -1){
            return -1;
        }
        int idx = list;
        while (idx!= -1 && array[idx].data != target){
            idx = array[idx].next;
        }
        if (idx==-1) {
            cout << "Target "<< target << " Not Found!!" << endl;
            return -1;
        }
        int post_idx = array[idx].next;
        array[index].next = post_idx;
        cout << "Just inserted after target("<< target <<"): " << endl;
        array[index].display();
        cout << endl;
        array[idx].next = index;
        return 0;
    }

    int del_beg(){
        if (list == -1){
            cout << "List underflow!!" << endl;
            return -1;
        }
        int ret_val = array[list].data;
        int tidx = list;
        list = array[list].next;

        cout << "Just deleted at beginning: " << endl;
        array[tidx].display();
        cout << endl;

        free_node(tidx);
```

```
        return ret_val;
    }

    int del_end(){
        if (list == -1){
            cout << "List underflow!!" << endl;
            return -1;
        }
        int pre_tidx = list;
        int tidx = list;
        while (array[tidx].next != -1){
            pre_tidx = tidx;
            tidx = array[tidx].next;
        }
        array[pre_tidx].next = -1;
        int ret_val = array[tidx].data;

        cout << "Just deleted at end: " << endl;
        array[tidx].display();
        cout << endl;

        free_node(tidx);
        return ret_val;
    }

    int del_after(int target){
        if (list == -1){
            cout << "List underflow!!" << endl;
            return -1;
        }
        int pre_tidx = list;
        while (pre_tidx!=-1 && array[pre_tidx].data != target){
            pre_tidx = array[pre_tidx].next;
        }
        if (pre_tidx==-1){
            cout << "Target("<< target << ") Not Found!!" << endl;
            return -1;
        }
        if (array[pre_tidx].next == -1) {
            cout << "No element after target: "<< target << "!" << endl;
            return -1;
        }
        int tidx = array[pre_tidx].next;
        int post_tidx = array[tidx].next;

        array[pre_tidx].next = post_tidx;
        int ret_val = array[tidx].data;

        cout << "Just deleted after node("<< target << "): " << endl;
        array[tidx].display();
        cout << endl;

        free_node(tidx);
        return ret_val;
```

```
    }

};

int main(){
    StaticList myList(5);
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.ins_beg(5);
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.ins_end(10);
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.ins_beg(6);
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.ins_end(15);
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.ins_end(15);
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.ins_specific(4, 6);
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.del_beg();
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.del_end();
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.del_after(15);
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.del_end();
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.del_end();
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.del_beg();
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();
    myList.del_beg();
    myList.traverse_display_active_list();
    myList.traverse_display_free_list();

    return 0;
}
```

Output:

```
pawanadhikari@Pawans-MacBook-Air 06_StaticList % gcr 01_StaticList.cpp
Active List:
[-1|-1]
--------------------------
Free List:
[0|1]--->[0|2]--->[0|3]--->[0|4]--->[0|-1]
.................................
Just inserted at beginning:
[5|-1]
Active List:
[5|-1]
--------------------------
Free List:
[0|2]--->[0|3]--->[0|4]--->[0|-1]
.................................
Just inserted at end:
[10|2]
Active List:
[5|1]--->[10|-1]
--------------------------
Free List:
[0|3]--->[0|4]--->[0|-1]
.................................
Just inserted at beginning:
[6|0]
Active List:
[6|0]--->[5|1]--->[10|-1]
--------------------------
Free List:
[0|4]--->[0|-1]
.................................
Just inserted at end:
[15|4]
Active List:
[6|0]--->[5|1]--->[10|3]--->[15|-1]
--------------------------
Free List:
[0|-1]
.................................
Just inserted at end:
[15|-1]
Active List:
[6|0]--->[5|1]--->[10|3]--->[15|4]--->[15|-1]
--------------------------
Free List:
[-1|-1]
.................................
List Overflow!!
Active List:
[6|0]--->[5|1]--->[10|3]--->[15|4]--->[15|-1]
--------------------------
Free List:
```

```
[-1|-1]
.................................
Just deleted at beginning:
[6|0]
Active List:
[5|1]--->[10|3]--->[15|4]--->[15|-1]
--------------------------
Free List:
[6|-1]
.................................
Just deleted at end:
[15|-1]
Active List:
[5|1]--->[10|3]--->[15|-1]
--------------------------
Free List:
[15|2]--->[6|-1]
.................................
No element after target: 15!
Active List:
[5|1]--->[10|3]--->[15|-1]
--------------------------
Free List:
[15|2]--->[6|-1]
.................................
Just deleted at end:
[15|-1]
Active List:
[5|1]--->[10|-1]
--------------------------
Free List:
[15|4]--->[15|2]--->[6|-1]
.................................
Just deleted at end:
[10|-1]
Active List:
[5|-1]
--------------------------
Free List:
[10|3]--->[15|4]--->[15|2]--->[6|-1]
.................................
Just deleted at beginning:
[5|-1]
Active List:
[-1|-1]
--------------------------
Free List:
[5|1]--->[10|3]--->[15|4]--->[15|2]--->[6|-1]
.................................
List underflow!!
Active List:
[-1|-1]
--------------------------
Free List:
```

```
[5|1]--->[10|3]--->[15|4]--->[15|2]--->[6|-1]
.....................................
```

Discussions

```
- We implemented the static list using a Node struct containing integer
type data and integer type index.
- The static list class itself has two pointers, list and avail.
- We initialised the list by filling the avail list.
- We also implemented methods to display both avail and active list.
- The methods for insertions and deletion for each case were also
implemented.
- Finally the impelmented was tested by defining a size 5 static lists and
performing insertions and deletions.
- It is clear from the output that edge cases (Overflow & Underflow) were
sucessfully handled.
- The growth and decay of active list and free list is exactly inverted.
i.e as free list decreases in size, active list increases suggesting
insertion operations happenning, and as active list decreases in size, free
list increases, suggesting deletion operations happening.
```

Conclusion:

```
Hence we can conclude that we have sucessfully studied, and understood
static linked lists or static lists in lab.
```